Lesson 3.1: DevOps

UMD DATA605 - Big Data Systems

Lesson 3.1: DevOps

Instructor: Dr. GP Saggese, gsaggese@umd.edu

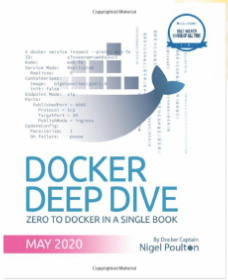
SCIENCE ACADEMY

1 / 17

2 / 17: Docker - Resources

Docker - Resources

- We will use Docker during the class project and most tutorials
- Concepts in the slides
- Class tutorials:
  - tutorial_docker
  - tutorial_docker_compose



- Web resources:
  - Docker Tutorial for beginners
  - https://labs.play-with-docker.com/
  - https://training.play-with-docker.com
  - A Beginner-Friendly Introduction to Containers, VMs and Docker
  - Official Docker Getting Started Tutorial
- Mastery:
  - Poulton, Docker Deep Dive: Zero to Docker in a single book, 2020

SCIENCE ACADEMY

2 / 17

- **Docker Usage in Class**:
  - Docker is a tool that allows us to create, deploy, and run applications in containers. We will be using Docker extensively during our class projects and tutorials. This means you'll get hands-on experience with Docker, which is a valuable skill in the tech industry.
- **Concepts in Slides**:
  - The slides will cover key Docker concepts. These concepts will help you understand how Docker works and how to use it effectively in your projects.
- **Class Tutorials**:
  - There are specific tutorials provided to help you learn Docker. The first tutorial, *tutorial_docker*, will introduce you to the basics of Docker. The second, *tutorial_docker_compose*, will teach you about Docker Compose, a tool for defining and running multi-container Docker applications.
- **Web Resources**:
  - Several online resources are available to supplement your learning. These include beginner-friendly tutorials and interactive labs where you can practice using Docker. The official Docker Getting Started Tutorial is a great place to begin if you're new to Docker.
- **Mastery**:
  - For those who want to dive deeper into Docker, the book "Docker Deep Dive: Zero to Docker in a single book" by Nigel Poulton is recommended. It provides a comprehensive guide to mastering Docker.

3 / 17: Application Deployment

Application Deployment

- For (almost all) Internet companies, **the application is the business**
  - If the application breaks, the business stops
  - E.g., Amazon, Google, Facebook, online banks, travel sites, ...
- **Problem**
  - Release, deploy, manage, monitor applications
- **Solutions**
  - Before 2000s: "bare-metal era"
  - 2000s-2010s: "virtual machine era"
  - After ~2013: "container era"

SCIENCE ACADEMY

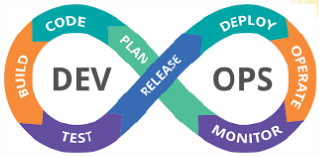
3 / 17

- **Application Deployment**
  - In the world of Internet companies, the application is essentially the backbone of the business. This means that if the application experiences any issues or downtime, it can directly impact the business operations. For example, companies like Amazon, Google, Facebook, online banks, and travel sites rely heavily on their applications to function smoothly. If their applications fail, it can lead to significant disruptions and potential financial losses.
- **Problem**
  - The main challenge here is how to effectively release, deploy, manage, and monitor these applications. This is crucial because any misstep in these processes can lead to application downtime or performance issues, which can negatively affect the business.
- **Solutions**
  - Over the years, the approach to application deployment has evolved significantly:
    - **Before 2000s: "bare-metal era"**: During this time, applications were deployed directly on physical servers. This approach was less flexible and more resource-intensive.
    - **2000s-2010s: "virtual machine era"**: This era introduced virtual machines, which allowed multiple applications to run on a single physical server, improving resource utilization and flexibility.
    - **After ~2013: "container era"**: Containers have become the preferred method for deploying applications. They offer even greater flexibility and efficiency by allowing applications to run in isolated environments, making it easier to manage and scale them.

4 / 17: DevOps

DevOps

- **DevOps** = practices combining:
  - Software development (*dev*)
  - IT operations (*ops*)
- **Containers revolutionized DevOps**
  - Enable independence between app development and IT ops
    - One team creates an app
    - Another team deploys and manages apps
  - Foster collaboration and innovation
    - IT: "It doesn't work!"
    - Devs: "What? It works for me"



- Plan
- Code
- Build
- Test
- Release
- Deploy
- Operate
- Monitor

SCIENCE ACADEMY

4 / 17

- **DevOps** is a set of practices that aims to bridge the gap between software development (*dev*) and IT operations (*ops*). The goal is to improve collaboration and productivity by automating workflows and continuously measuring application performance.
- **Containers** have significantly impacted DevOps by allowing applications to be packaged with all their dependencies, making them portable and consistent across different environments. This means that developers can focus on writing code, while operations teams can handle deployment and management without worrying about compatibility issues. This separation fosters better collaboration and innovation, as it reduces the classic blame game where IT might say, "It doesn't work!" and developers respond with, "What? It works for me!"
- The image on the right illustrates the DevOps lifecycle, which includes stages like planning, coding, building, testing, releasing, deploying, operating, and monitoring. Each stage is crucial for ensuring that software is delivered quickly and reliably, with continuous feedback loops to improve the process.

5 / 17: Run on bare metal

Run on bare metal

- < 2000s
  - Running few applications per server (no virtualization)
- Pros
  - No virtualization overhead
- Cons
  - Insecure due to no application separation
  - Expensive
- Expensive / low efficiency
  - New server for each application
  - Hard to spec machines → buy "big and fast servers"
    - Overpowered servers at 5-10% capacity
    - Significant spending during 2000 DotCom boom on machines and networks
- Resurfaced in 2020 with different Cloud Computing use cases
- Winners: Cisco, Sun, Microsoft

SCIENCE ACADEMY

5 / 17

- **< 2000s**
  - During this time, servers were used to run only a few applications, and there was no virtualization. This means that each server was dedicated to specific tasks without sharing resources with other applications.
- **Pros**
  - The main advantage of this approach was the absence of virtualization overhead. Virtualization can slow down performance because it adds an extra layer of software between the hardware and the applications. Without it, applications could run faster.
- **Cons**
  - A major downside was the lack of security. Without virtualization, there was no separation between applications, which could lead to security vulnerabilities if one application was compromised.
  - It was also expensive because each application required its own server, leading to high costs.
- **Expensive / low efficiency**
  - Organizations had to purchase a new server for each application, which was not cost-effective.
  - It was challenging to determine the right specifications for servers, often resulting in buying "big and fast servers" that were more powerful than needed. These servers typically operated at only 5-10% of their capacity, leading to wasted resources.
  - During the 2000 DotCom boom, companies spent a lot on these overpowered machines and the networks to support them.
- **Resurfaced in 2020 with different Cloud Computing use cases**
  - The concept of running on bare metal re-emerged in 2020, but with new applications

in cloud computing. This suggests that while the approach was initially inefficient, it found new relevance in modern technology.

- **Winners**: Cisco, Sun Microsystems, Microsoft
  - These companies benefited from the demand for powerful servers and networking equipment during this period. They were key players in the industry, providing the necessary technology to support the infrastructure needs of the time.

6 / 17: Virtual Machine Era

Virtual Machine Era

- **Circa 2000-2010: Virtual Machine**
  - Run multiple OS copies on the same hardware
- **Pros**
  - Run multiple apps safely on one server
  - Use existing servers with spare capacity
- **Cons**
  - Each VM needs an OS (wastes CPU, RAM, disk)
  - Buy OS license
  - Monitor and patch each OS
  - Slow boot times
- **Winners**: VMWare, RedHat, Citrix

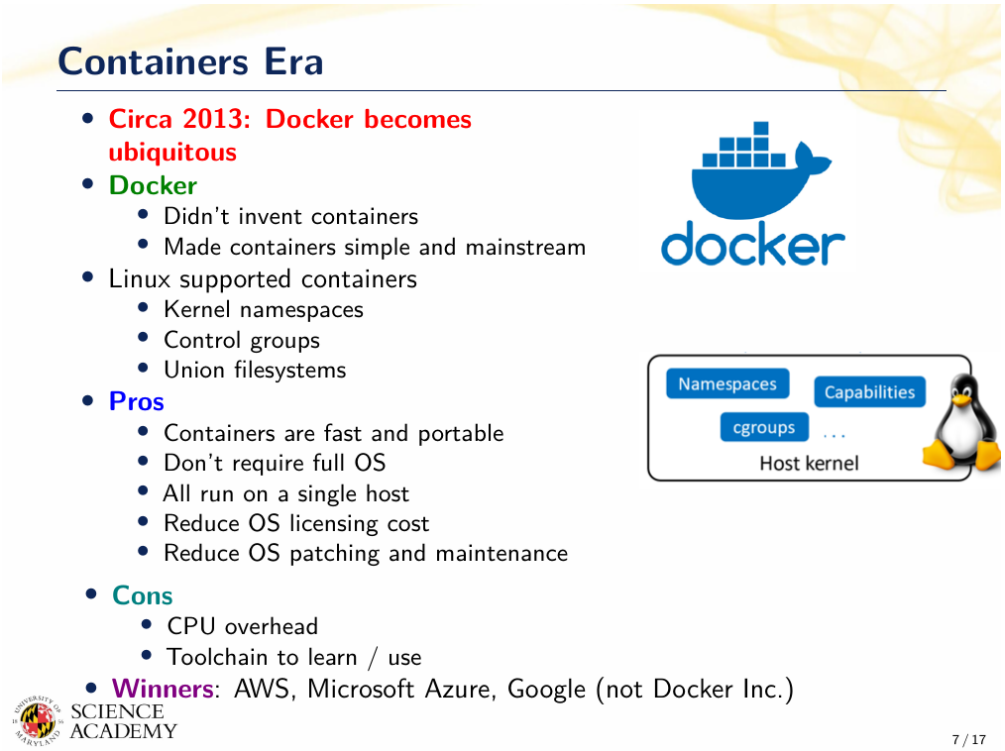
SCIENCE ACADEMY

6 / 17

- **Circa 2000-2010: Virtual Machine**
  - During this period, the concept of a *Virtual Machine (VM)* became popular. A VM allows you to run multiple operating systems on a single physical machine. This means you can have different environments on the same hardware, which was a big deal back then.
- **Pros**
  - **Run multiple apps safely on one server**: VMs provide isolation, meaning if one application crashes, it doesn't affect others. This is great for running different applications on the same server without them interfering with each other.
  - **Use existing servers with spare capacity**: Instead of buying new hardware, you can use the extra capacity of your current servers to run more applications, saving costs and resources.
- **Cons**
  - **Each VM needs an OS (wastes CPU, RAM, disk)**: Every VM requires its own operating system, which uses up a lot of resources like CPU, memory, and storage. This can be inefficient.
  - **Buy OS license**: You need to purchase a separate license for each operating system running on a VM, which can be expensive.
  - **Monitor and patch each OS**: Each operating system needs to be maintained, which includes monitoring for issues and applying updates or patches. This can be time-consuming.
  - **Slow boot times**: Starting up a VM can take a while because it involves booting up an entire operating system, which can be slow.
- **Winners**: VMWare, RedHat, Citrix

- These companies were leaders in the VM space. They developed technologies and solutions that made it easier to implement and manage virtual machines, gaining significant market share during this era.

7 / 17: Containers Era



- **Circa 2013: Docker becomes ubiquitous**
  - Around 2013, Docker started to gain widespread popularity. It became a key player in the tech industry by making container technology accessible and easy to use for developers and organizations.
- **Docker**
  - Docker didn't create the concept of containers, but it revolutionized how they were used. By simplifying the process, Docker made it possible for more people to adopt container technology, which led to its mainstream success.
- **Linux supported containers**
  - Containers rely on several Linux features:
    * *Kernel namespaces* provide isolation for processes, ensuring they don't interfere with each other.
    * *Control groups* manage and limit the resources that containers can use, like CPU and memory.
    * *Union filesystems* allow multiple file systems to be layered, which is essential for container efficiency.
- **Pros**
  - Containers are known for being fast and portable, meaning they can run consistently across different environments.
  - They don't need a full operating system, which saves resources.
  - Multiple containers can run on a single host, optimizing hardware usage.
  - By using containers, companies can save on operating system licensing costs and reduce the need for frequent OS updates and maintenance.
- **Cons**

  - Despite their benefits, containers can introduce some CPU overhead, which might affect performance.
  - There is a learning curve associated with the toolchain required to effectively use and manage containers.
- **Winners**: AWS, Microsoft Azure, Google (not Docker Inc.)
  - While Docker popularized containers, the real beneficiaries were cloud service providers like AWS, Microsoft Azure, and Google. They integrated container technology into their services, offering scalable solutions to businesses. Docker Inc., despite its pioneering role, didn't capture as much of the market share in terms of revenue.

8 / 17: Serverless Computing

Serverless Computing

- Containers run in an OS, OS runs on a host
  - Where is the host running?
    - Local (your laptop)
    - On premise (your computer in a rack)
    - Cloud instance (e.g., AWS EC2)
  - How is the host running?
    - On bare-metal server
    - On a virtual machine
    - On a virtual machine running a virtual machine
- Serverless computing
  - Application runs without concern for *"how"* or *"where"*
  - E.g., AWS Lambda

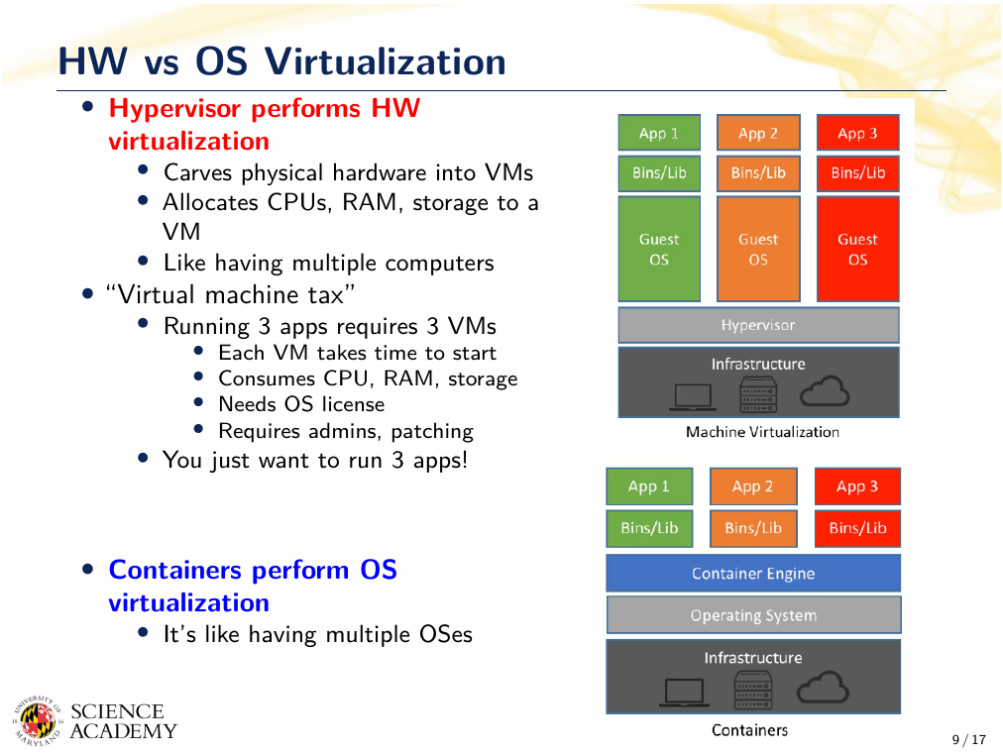
SCIENCE ACADEMY

8 / 17

- **Serverless Computing**
- **Containers run in an OS, OS runs on a host**
  - This point is about understanding the layers involved when running applications. **Containers** are a way to package applications so they can run consistently across different environments. They rely on an **Operating System (OS)** to function, and this OS needs a **host** to run on.
  - **Where is the host running?**
    - *Local (your laptop)*: This means the host is your personal computer. It's convenient for development and testing.
    - *On-premise (your computer in a rack)*: This refers to servers physically located within your organization. It's often used for sensitive data or specific compliance needs.
    - *Cloud instance (e.g., AWS EC2)*: Here, the host is a virtual server provided by a cloud service like Amazon Web Services. It's flexible and scalable.
  - **How is the host running?**
    - *On a bare-metal server*: This is a physical server without any virtualization. It's used for high-performance needs.
    - *On a virtual machine*: This is a software-based emulation of a physical computer. It allows multiple OS instances on a single physical machine.
    - *On a virtual machine running on a virtual machine*: This is a more complex setup, often used for testing or specific use cases.
- **Serverless computing**

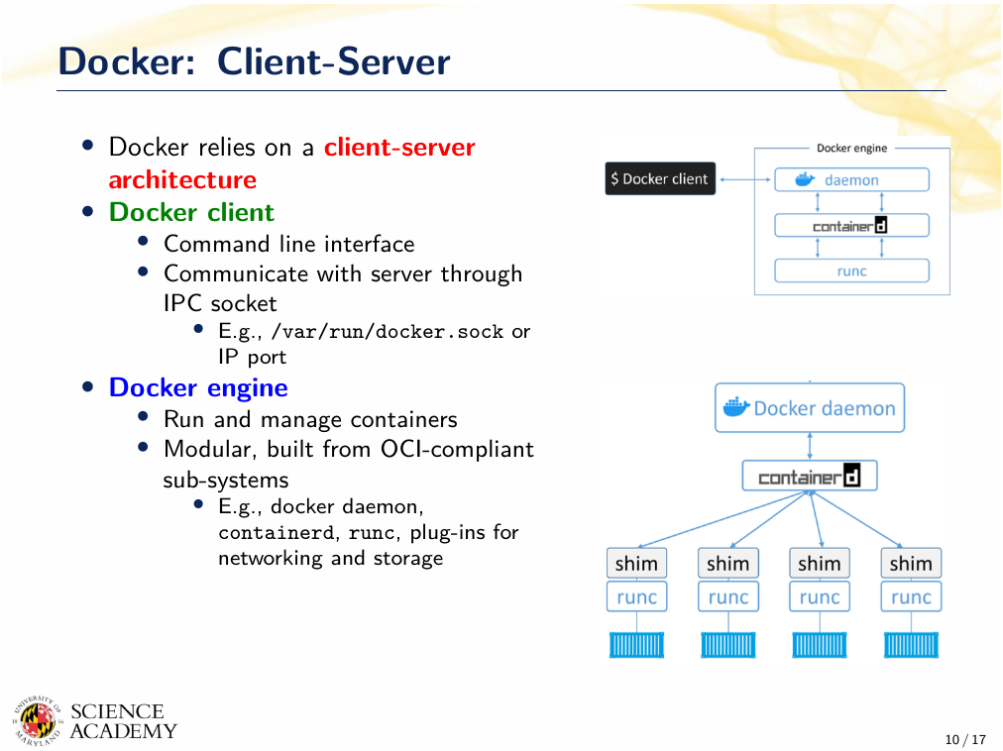
- This concept allows you to run applications without worrying about the underlying infrastructure. You don't need to manage servers or worry about scaling.
- *E.g., AWS Lambda*: This is a popular serverless service where you can run code in response to events without provisioning or managing servers. It's cost-effective and simplifies deployment.

9 / 17: HW vs OS Virtualization



- **Hypervisor performs HW virtualization**
  - A *hypervisor* is a technology that allows you to create and manage virtual machines (VMs) by dividing up the physical hardware. This means you can run multiple VMs on a single physical server, each acting like a separate computer.
  - The hypervisor allocates resources such as CPUs, RAM, and storage to each VM. This is similar to having multiple computers, each with its own dedicated resources, but all running on the same physical machine.
- **"Virtual machine tax"**
  - When you want to run multiple applications using VMs, each application typically requires its own VM. This can be inefficient because:
    * Each VM takes time to start up, which can delay application deployment.
    * VMs consume significant resources like CPU, RAM, and storage, even if the applications themselves don't need much.
    * Each VM needs its own operating system license, which can be costly.
    * Managing multiple VMs requires administrative effort and regular updates or patches, adding to the workload.
  - This setup can be overkill if you simply want to run a few applications without the overhead of managing multiple VMs.
- **Containers perform OS virtualization**
  - Containers offer a more efficient alternative by allowing multiple isolated environments to run within the same operating system. This means you can run several applications without the need for separate VMs, reducing resource usage and simplifying management. Containers share the same OS kernel but keep applications isolated, providing a lightweight and fast solution for running multiple applications.

10 / 17: Docker: Client-Server

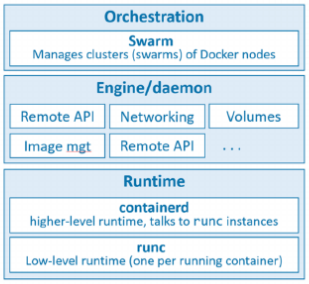


- **Docker relies on a client-server architecture**
  - Docker uses a system where the client and server work together to manage containers. This setup allows users to interact with Docker through commands, while the server handles the heavy lifting of running and managing the containers.
- **Docker client**
  - The Docker client is essentially the command line interface (CLI) that users interact with. When you type commands into your terminal, you're using the Docker client.
  - It communicates with the Docker server using an Inter-Process Communication (IPC) socket. This is a special file or network port that allows the client and server to exchange information. Common examples include `/var/run/docker.sock` for local communication or an IP port for remote communication.
- **Docker engine**
  - The Docker engine is the core part of Docker that actually runs and manages the containers. It's like the brain of Docker, handling all the tasks needed to keep containers running smoothly.
  - It's built in a modular way, meaning it's made up of different parts that can work together. These parts are compliant with the Open Container Initiative (OCI) standards, ensuring compatibility and flexibility. Key components include the Docker daemon, `containerd` for managing container lifecycles, `runc` for running containers, and various plugins for networking and storage. This modularity allows Docker to be both powerful and adaptable to different needs.

11 / 17: Docker Architecture

Docker Architecture

- **Docker run-time**
  - `runc`: start and stop containers
  - `containerd`
    - Pull images
    - Create containers, volumes, network interfaces
- **Docker engine**
  - `dockerd`
    - Expose remote API
    - Manage images, volumes, networks
- **Docker orchestration**
  - `docker swarm`
  - Manage clusters of nodes
  - Replaced by Kubernetes
- **Open Container Initiative (OCI)**
  - Standardize low-level components of container infrastructure
  - E.g., image format, run-time API
  - "Death" of Docker



SCIENCE ACADEMY

11 / 17

- **Docker runtime**
  - **`runc`**: This is a lightweight tool that is responsible for starting and stopping containers. Think of it as the component that directly interacts with the container's operating system to manage its lifecycle.
  - **`containerd`**: This is a more comprehensive tool that handles several tasks. It pulls images from repositories, creates containers, and manages resources like volumes and network interfaces. Essentially, it acts as the backbone for container management.
- **Docker engine**
  - **`dockerd`**: This is the core service that runs on a host machine. It exposes a remote API, which allows users to interact with Docker through commands. It also manages images, volumes, and networks, making it the central hub for Docker operations.
- **Docker orchestration**
  - **`docker swarm`**: This tool is used to manage clusters of nodes, allowing for the deployment and scaling of applications across multiple machines. However, it's important to note that many users now prefer Kubernetes for orchestration due to its advanced features and flexibility.
- **Open Container Initiative (OCI)**
  - This initiative aims to standardize the low-level components of container infrastructure, such as the image format and runtime API. It represents the evolution of Docker standards, ensuring compatibility and interoperability across different container technologies. This standardization is crucial for the broader adoption and integration of container technologies in various environments.

12 / 17: Docker Container

Docker Container

- **Docker Container**
  - Unit of computation
  - Lightweight, stand-alone, executable software package
- **Includes everything needed to run**
  - Code
  - Runtime/system libraries
  - Settings
- **Run-time object**
  - Docker images are build-time objects
  - Like program running (container) vs program code (image)

SCIENCE ACADEMY

12 / 17

- **Docker Container**
  - *Unit of computation*: Think of a Docker container as a small, self-contained unit that can perform a specific task or run a particular application. It's like a virtual box that holds everything needed to execute a piece of software.
  - *Lightweight, stand-alone, executable software package*: Containers are designed to be efficient and portable. They don't require a full operating system to run, which makes them much lighter than traditional virtual machines. This means they can be easily moved and run on different systems without compatibility issues.
- **Includes everything needed to run**
  - *Code*: The actual application or software that you want to run is included in the container.
  - *Runtime/system libraries*: All the necessary libraries and dependencies that the application needs to function are bundled within the container. This ensures that the application runs the same way regardless of where the container is deployed.
  - *Settings*: Configuration files and environment settings are also part of the container, ensuring consistent behavior across different environments.
- **Run-time object**
  - *Docker images are build-time objects*: A Docker image is like a blueprint or a recipe. It contains all the instructions needed to create a container. You build an image once and then use it to create multiple containers.
  - *Like program running (container) vs program code (image)*: Think of a Docker image as the static code of a program, while a Docker container is the running instance of that program. Just like you can have multiple instances of a program running on your computer, you can have multiple containers running from the same image.

13 / 17: Docker Image

Docker Image

- **Docker Image**
  - Unit of deployment
- **Contains everything needed to run an app**
  - Application code
  - Application dependencies
  - Minimal OS support
  - File system
- Users can
  - Build images from Dockerfiles
  - Pull pre-built images from a registry
- Multiple layers stacked
  - Typically few 100s MBs

SCIENCE ACADEMY

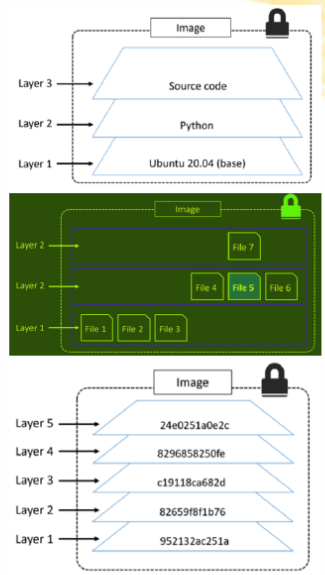
13 / 17

- **Docker Image**
  - A *Docker Image* is essentially a package that contains everything needed to run a software application. Think of it as a blueprint for creating a running instance of your application, known as a container. This makes it a crucial unit of deployment in the world of containerization.
- **Contains everything needed to run an app**
  - **Application code**: This is the actual codebase of the application you want to run.
  - **Application dependencies**: These are the libraries and tools your application needs to function properly.
  - **Minimal OS support**: Docker images include just enough of an operating system to run the application, which helps keep them lightweight.
  - **File system**: The image contains a complete file system that the application can use.
- **Users can**
  - **Build images from Dockerfiles**: A Dockerfile is a script that contains a series of instructions on how to build a Docker image. Users can write these files to customize their images.
  - **Pull pre-built images from a registry**: Docker Hub is a popular registry where users can find and download pre-built images, saving time and effort.
- **Multiple layers stacked**
  - Docker images are made up of multiple layers, each representing a set of changes or additions. These layers are stacked on top of each other, and this structure helps in efficient storage and transfer. Typically, these images are a few hundred megabytes in size, balancing comprehensiveness and efficiency.

14 / 17: Docker Image Layers

Docker Image Layers

- **Docker image** is a configuration file listing layers and metadata
  - Composed of read-only layers
  - Each layer is independent
  - Each layer comprises many files
- **Docker driver**
  - Stacks layers as a unified filesystem
  - Implements copy-on-write behavior
  - Files from top layers can obscure files from bottom layers
- **Layer hash**
  - Each layer has a hash based on content
  - Layers are pulled and pushed compressed
- **Image hash**
  - Each image has a hash
  - Hash is a function of the config file and layers
  - Image changes generate a new hash



SCIENCE ACADEMY

14 / 17

- **Docker image** is essentially a blueprint for creating containers. It is made up of multiple *read-only layers*, each of which is a snapshot of the filesystem at a certain point. These layers are independent, meaning changes in one layer do not affect others. Each layer can contain numerous files, and together they form the complete image.
- **Docker driver** plays a crucial role in managing these layers. It stacks them to create a single, unified filesystem that the container uses. This driver also implements a *copy-on-write* mechanism, which means that when a file is modified, a copy is made in the top layer, leaving the original file unchanged. This allows for efficient storage and quick changes without altering the base layers.
- **Layer hash** is a unique identifier for each layer, generated based on its content. This hash ensures that layers are consistent and can be easily shared or reused. When layers are transferred between systems, they are compressed to save bandwidth and storage space.
- **Image hash** is another unique identifier, but for the entire Docker image. It is derived from the configuration file and all the layers. Any modification to the image, such as adding a new layer or changing the configuration, results in a new hash. This ensures that each version of an image is distinct and traceable.

15 / 17: Docker: Container Data

Docker: Container Data

- A container has access to different data
- **Container storage**
  - Copy-on-write layer in the image
  - Ephemeral (temporary data)
  - Data persists until the container is killed
    - Stopping or pausing a container doesn't lose data
  - Containers are immutable
    - Avoid writing persistent data into containers
- **Bind-mount a local dir**
  - Mount a local dir to a dir inside a container
- **Docker volumes**
  - Volumes exist separately from the container
    - E.g., store Postgres DB content
  - State is permanent across container invocations
  - Shareable across containers

SCIENCE ACADEMY

15 / 17

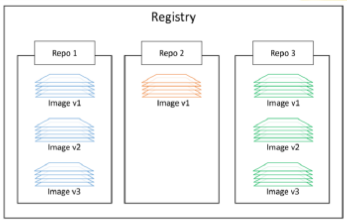
- **Docker: Container Data**
  - When using Docker, it's important to understand how data is managed within containers. Containers can access different types of data storage, each with its own characteristics and use cases.
- **Container storage**
  - Containers use a *copy-on-write* layer, which means that any changes made to the container's filesystem are stored in a separate layer on top of the original image. This allows for efficient use of storage and quick deployment.
  - This storage is *ephemeral*, meaning it's temporary. The data will persist as long as the container exists, but once the container is removed, the data is lost.
  - It's important to note that stopping or pausing a container does not result in data loss; the data remains intact until the container is explicitly removed.
  - Containers are designed to be *immutable*, which means they should not be used for storing persistent data. Instead, they should be treated as disposable and easily replaceable.
- **Bind-mount a local dir**
  - This method allows you to link a directory on your local machine to a directory inside the container. This is useful for scenarios where you need to access or modify files on your host system from within the container.
- **Docker volumes**
  - Volumes are a more robust solution for data persistence. They exist independently of the container lifecycle, meaning they are not tied to the container's existence.
  - For example, you might use a volume to store the data of a PostgreSQL database, ensuring that the data remains available even if the container is deleted or recreated.
  - Volumes are *permanent* and can be shared across multiple containers, making them ideal

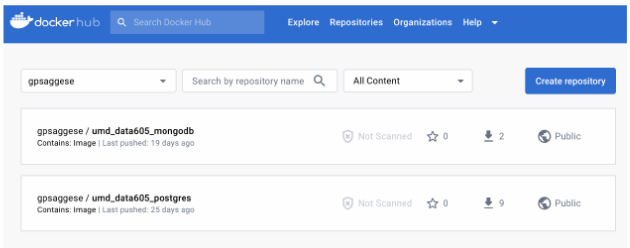
for scenarios where data needs to be accessed by different containers or retained across container restarts.

16 / 17: Docker Repos

Docker Repos

- **Docker Repo (Registry)**
  - E.g., DockerHub, AWS ECR
  - Store Docker images
    - `<registry>/<repo>:<tag>`
    - E.g., `docker.io/alpine:latest`
  - Some repos are vetted by Docker
  - Unofficial repos shouldn't be trusted
  - E.g., https://hub.docker.com/





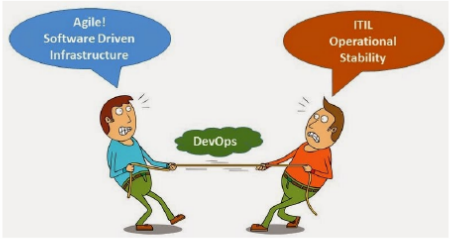
SCIENCE ACADEMY

16 / 17

- **Docker Repo (Registry)**
  - *Docker repositories*, often referred to as registries, are platforms where Docker images are stored and managed. These images are essential for deploying applications in a consistent environment.
  - **Examples**: Popular registries include DockerHub and AWS Elastic Container Registry (ECR). DockerHub is the most widely used public registry, while AWS ECR is a service provided by Amazon for storing Docker images in the cloud.
  - **Store Docker images**: Images are stored in a structured format using the pattern `<registry>/<repo>:<tag>`. For instance, `docker.io/alpine:latest` refers to the Alpine Linux image stored on DockerHub with the 'latest' tag, indicating the most recent version.
  - **Vetted Repos**: Some repositories are officially vetted by Docker, ensuring they meet certain standards for security and reliability. These are generally safer to use.
  - **Unofficial Repos**: Be cautious with unofficial repositories, as they might not be secure or reliable. It's important to verify the source before using these images to avoid potential security risks.
  - **Example URL**: DockerHub can be accessed at https://hub.docker.com/, where users can search for and download Docker images.

17 / 17: Devops = Devs + Ops

Devops = Devs + Ops



- **Devs**
- Implement app
  - Python, virtual env
- Containerize app
  - Create Dockerfile
  - Instructions to build image
- Build image
- Run app as container
- Test locally

- **Ops**
- Download container images
  - Filesystem, application, dependencies
- Start / destroy containers
- Reproduce issues easily
  - "Here is the log"
  - Run command line
  - Deploy on test system and debug

SCIENCE ACADEMY

17 / 17

- **Devs**
  - *Implement app*: Developers are responsible for writing the code for the application. They often use programming languages like Python and set up a virtual environment to manage dependencies and packages specific to the project.
  - *Containerize app*: This involves creating a Dockerfile, which is a script with instructions on how to build a Docker image. The Dockerfile specifies everything needed to run the app, including the base image, dependencies, and any necessary configurations.
  - *Build image*: Once the Dockerfile is ready, developers build the Docker image. This image is a snapshot of the app and its environment, ensuring consistency across different systems.
  - *Run app as container*: Developers test the application by running it inside a container. This helps in isolating the app from the host system, making it easier to manage and deploy.
  - *Test locally*: Before deploying the app to production, developers test it locally to ensure it works as expected and to catch any bugs early.
- **Ops**
  - *Download container images*: Operations teams handle the infrastructure side, starting with downloading the necessary container images. These images include the filesystem, application, and all dependencies required to run the app.
  - *Start / destroy containers*: Ops teams manage the lifecycle of containers, including starting and stopping them as needed. This helps in scaling applications and managing resources efficiently.
  - *Reproduce issues easily*: One of the key benefits of using containers is the ability to reproduce issues consistently. Ops teams can share logs and run command-line tools to

diagnose problems. They can also deploy the app on a test system to debug and resolve issues before they affect users.